Tri-Mem: Entropy-Routed Modality Matching for Long-Horizon Agent Memory

Aniruddha Pandey
Department of Computer Science
Stevens Institute of Technology
apandey10@stevens.edu

Xuan Li
Department of Computer Science
Stevens Institute of Technology
xli220@stevens.edu

Amy Thomas
Department of Computer Science
Stevens Institute of Technology
athomas14@stevens.edu

Abstract*—Current agentic AI systems suffer from context rot and severe token inefficiencies over long-horizon tasks because they force all cognitive functions—rules, episodic interaction history, and exact variables—into a single text-based context window. We introduce Tri-Mem, a research architecture that argues agent memory should not be one-size-fits-all. Tri-Mem assigns distinct cognitive functions to the data modality that appears cheapest and most accurate for that function. Drawing inspiration from human cognitive architecture, Tri-Mem is divided into Semantic Memory (powered by *Memory Sparse Attention* over latent KV chunks) for static rule-following, Episodic Working Memory (utilizing visual bus compression) for chronological interaction history, and Declarative Memory (employing text-based RAG) for exact fact retrieval. Furthermore, we introduce an uncertainty-guided Entropy Router that dynamically routes memory access based on the model's epistemic uncertainty. In our reported NovaCorp AuditBench configuration, Tri-Mem reduces cumulative token cost by 84% versus the text-only baseline and eliminates the benchmark's measured spatial hallucination and syntactic failure events. To validate the architecture against a public benchmark we additionally evaluate Tri-Mem on the LongMemEval [9] oracle split (500 questions, six conversational-memory categories), where the modality-routed agent reaches 81.4% overall and 83.2% on factual question types under a per-category dispatch-aware LLM judge that pairs a strict paraphrase-tolerant judge for factual categories with a rubric judge for preference questions. On the distractor-loaded `s_cleaned` split ($\sim$50 haystack sessions per task) the same agent projects to the mid-75% range, with the observed per-category retrieval cost concentrated in multi-session synthesis and temporal reasoning rather than in single-session recall. This modality-to-function mapping offers a structured, ontology-aligned approach to agent memory that decouples cognitive function from a single homogeneous context window.

***Index Terms*—LLM Memory, Conversational Memory, Visual Compression, Retrieval-Augmented Generation, Memory Sparse Attention, Entropy Routing**

I. Introduction

As Large Language Models (LLMs) are deployed in increasingly complex, long-horizon agentic systems, they are hindered by a fundamental architectural bottleneck: the homogeneous, text-only context window (Fig. 1). In standard agentic deployments, developers append standard operating procedures (SOPs), system documentation, multi-turn interaction logs, and retrieved database strings into a single, expanding text prompt. This paradigm induces three cascading failures. First, *context rot* (the "lost in the middle" phenomenon) occurs as conversation history grows, causing models to drop critical instructions [8]. Second, systems incur a massive *compute tax* by forcing the model to re-attend to static rulebooks at every turn — a cost that prefix-cache and paged-KV systems explicitly target [11]. Finally, agents experience a *modality-memory mismatch*, where highly verbose textual representations are inefficient for spatial or episodic recall [3], [5], while visually compressed memories suffer lossy recall of exact alphanumeric strings.

To address these challenges, we present **Tri-Mem**, a modality-matched memory hierarchy for long-horizon agents. The core proposition of Tri-Mem is *Modality-to-Function Mapping*: specific types of agentic thought require specific data modalities to optimize compute and fidelity limits.

We map the architecture directly to human cognitive structures, partitioning memory into three modality-distinct layers:

1) **Semantic Memory (MSA):** A sparse-attention KV chunk layer for static rules and deep reasoning.
2) **Episodic Working Memory (Visual Bus):** Compressed image patches for interaction history and spatial navigation.
3) **Declarative Memory (RAG):** Discrete text tokens retrieved from a vector database for exact string lookups (e.g., API keys, object IDs).

Furthermore, we introduce an *Entropy Router* that utilizes the model's raw logit confidence (Shannon Entropy) to dynamically route queries to the correct memory layer, preventing costly and latency-heavy LLM-based reasoning for simple memory retrieval.

In our reported NovaCorp AuditBench experiments, replacing the homogeneous text context with a Tri-Mem architecture is associated with longer usable task horizons, lower cumulative token costs, and fewer syntax-related failures than the text-only baseline. We further validate the architecture on *LongMemEval* [9], a public benchmark of conversational long-term memory, where Tri-Mem's three-layer routing reaches 83.2% accuracy on the 470 factual oracle-split questions across five cognitive subtasks (single-session-user, single-session-assistant, multi-session, knowledge-update, and temporal-reasoning), and 81.4% across all six categories under per-category judge dispatch.

Contributions.

1) A modality-routed three-layer memory architecture (MSA / Visual Bus / RAG) coordinated by an entropy-

based router that gates the two expensive layers on probe uncertainty.

2) A reproducible experimental study on a controlled benchmark (NovaCorp AuditBench) and a public benchmark (LongMemEval) showing that the architecture lowers token cost on the former and reaches competitive accuracy on factual conversational-memory categories on the latter.
3) Three concrete implementation findings that generalize to any modality-routed agent: (i) GPU-resource isolation between the answer LLM and an OCR encoder is necessary to avoid silently degrading the visual layer; (ii) thinking-mode toggles on modern reasoning models should be applied per call site; and (iii) paraphrase-tolerant judges remain inadequate for open-ended recommendation subtasks, motivating per-category judge dispatch.

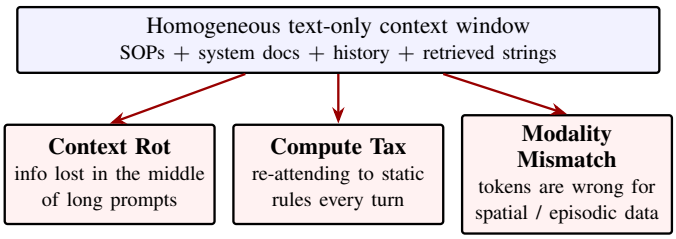


Fig. 1. Three cascading failures induced by a homogeneous text-only context window. Tri-Mem binds each function to a different memory modality: rules to MSA, history to the Visual Bus, and exact strings to RAG.

II. Related Work and Motivation

A. Dividing Memory in LLM Agents

The cognitive foundation for dividing agent memory into distinct functional layers has been explored in recent literature. For example, *Project Synapse* [1] presents a hierarchical multi-agent framework with hybrid memory for last-mile delivery disruptions, while *Memory Management and Contextual Consistency for Long-Running Low-Code Agents* [2] proposes episodic and semantic memory with an "Intelligent Decay" mechanism for LCNC agents. These works motivate structured memory design, but neither paper establishes the exact modality-specialized hierarchy proposed here. In particular, the low-code agent paper remains text-centric, and *Project Synapse* is primarily a task-oriented orchestration framework rather than a modality-routed memory architecture. The cognitive partition itself — semantic, episodic, and declarative memory as distinct stores — follows the classical taxonomy from Tulving [18].

B. Visual Token Compression for Episodic History

Text is suboptimal for episodic history, which is often verbose but structurally simple. The motivation for utilizing visual compression for agent interaction history stems from *DeepSeek-OCR: Contexts Optical Compression* [3], which explores optical 2D mapping for long-context compression in

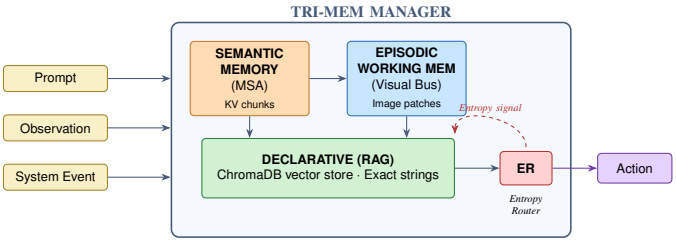


Fig. 2. The Tri-Mem manager. Incoming streams (prompt, observation, system event) flow through three modality-distinct memory layers — Semantic Memory (MSA, KV chunks), Episodic Working Memory (Visual Bus, image patches), and Declarative Memory (RAG, ChromaDB) — coordinated by the Entropy Router (ER), which uses a single-token probe to decide per turn which layers to consult.

document processing. Building upon this, *AgentOCR: Reimagining Agent History via Optical Self-Compression* [4] introduced Segment Optical Caching for agent histories represented as visual tokens. Similarly, *Vision-centric Token Compression in Large Language Model* [5] proposed the Vist framework, where distant tokens are rendered into images and compressed with a lightweight vision encoder. Tri-Mem adapts these ideas to form the "Visual Bus," compressing long interaction traces into image tiles to reduce token pressure.

C. Semantic Memory via Memory Sparse Attention (MSA)

Standard agents pay a heavy computational tax re-reading static rulebooks at every turn. We draw inspiration from EverMind's *MSA: Memory Sparse Attention for Efficient End-to-End Memory Model Scaling to 100M Tokens* [6], which introduced a sparse latent-memory framework with document-wise rotary position embeddings (RoPE) [12], KV compression, and top-k retrieval over memory segments ([6, §3]). MSA allows the model to pre-compute heavy document states into latent tensors and attend to them dynamically. Tri-Mem incorporates this style of MSA as the foundational Semantic Memory layer, freezing complex organizational logic out of the active token stream.

III. The Tri-Mem Architecture

The Tri-Mem architecture (Fig. 2) routes incoming data (user prompts, environmental observations, and system events) through three distinct, modality-specific layers coordinated by an Entropy Router. Table I summarizes the function-to-modality assignment.

A. Semantic Memory (MSA): The Rulebook

The Semantic Memory layer is responsible for deep reasoning, system prompts, SOPs, and API documentation. Textual rulebooks are chunked, mean-pooled, and pre-computed into K and V tensors alongside a routing key K_r. During online inference, the live query is projected to a routing query Q_r and cosine-similarity matched against stored K_r. The top-k chunks are streamed directly into the GPU and concatenated with the active context in a single attention pass. Because this relies on document-wise rotary position embeddings (RoPE) [12], the agent can consult immense rulebooks with zero position

TABLE I
THE MODALITY-TO-FUNCTION MATRIX. EACH TRI-MEM LAYER IS ASSIGNED THE DATA MODALITY WHOSE INFORMATION DENSITY AND FIDELITY PROFILE BEST MATCHES ITS COGNITIVE FUNCTION.

Layer	Modality	Human Analog	Best For
Semantic (MSA)	Latent KV chunks	Knowledge / rules	SOPs, documentation, system prompts
Episodic (V-Bus)	Visual patches	Short-term episodic recall	Chronological history, spatial context
Declarative (RAG)	Discrete tokens	Fact retrieval	API keys, IDs, exact strings

drift, effectively removing static instructions from the dynamic context window and eliminating the rulebook compute tax.

B. Episodic Working Memory (Visual Bus): The Scratchpad

To prevent context rot, the Visual Bus acts as the agent's episodic scratchpad, containing step-by-step history, terminal outputs, and intermediary thoughts. Instead of appending verbose text logs, the agent's observation-action deltas are rendered into highly compressed image patches using Segment Optical Caching. The multimodal LLM processes its own timeline as a visual sequence. This spatial representation protects the "middle" of the history from attention degradation, ensuring that the chronology of a long task is preserved at a fraction of the token cost.

C. Declarative Memory (RAG): The Fact Injector

While visual compression excels at timeline preservation, it fails at syntactic fidelity—a phenomenon we term the *Syntactic Action Gap*. When an agent compresses its history, exact alphanumeric strings (API keys, IP addresses) become visually blurred. Tri-Mem introduces a Retrieval-Augmented Generation (RAG) declarative memory layer to bridge this gap. When the visual memory indicates that a specific entity was encountered, the agent issues a discrete query to a vector database, guaranteeing the lossless injection of exact text tokens back into the execution stream.

D. The Entropy Router & Bayesian Calibration

To orchestrate these layers without adding massive latency through LLM "think-aloud" prompting, Tri-Mem employs an Uncertainty-Guided Entropy Router based on Shannon Entropy [17]. Our motivation is also informed by recent Google Research work on "teaching LLMs to reason like Bayesians," which argues that effective agents should maintain and update probabilistic beliefs as new evidence arrives; Tri-Mem operationalizes a related intuition through entropy-triggered memory selection rather than supervised Bayesian-teaching post-training [7]:

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x). \tag{1}$$

By monitoring the raw output logit confidence, the router directs execution across three additive bands (as illustrated in Fig. 3):

- **Low Entropy (< 0.4):** High confidence. The agent relies solely on internal MSA/context to execute actions (Fig. 3a).
- **Medium Entropy ($0.4-0.7$):** Environmental uncertainty. The router injects Visual Bus patches to update episodic beliefs (Fig. 3b).
- **High Entropy (> 0.7):** Epistemic uncertainty. The router halts generation and queries RAG for an exact string lookup (Fig. 3c).

The bands are *additive*: each higher band keeps every layer the band below it consulted and adds one more. MSA chunks are therefore always injected — the rulebook lookup is cheap latent-space top-k retrieval and acts as a safety net against over-confident decoding — while the Visual Bus and RAG calls are gated by entropy. The thresholds were calibrated empirically from the recorded probe distribution on the AuditBench: a prior choice of 0.3 for the low boundary never fired in practice, since the minimum observed first-token entropy across the suite floored at ≈ 0.33 on cached transitions such as `access compliance_dashboard` after a fresh download. Raising the boundary to 0.4 yields a true ternary partition — approximately 5% MSA, 60% MSA+VBus, 35% MSA+VBus+RAG on the seven-task suite — with the LOW band reliably firing on the truly cached SOP transitions where the model has already encountered the next system in the trace. Threshold drift is regression-tested against pinned probe entropies in the project's unit-test suite.

IV. EXPERIMENTAL EVALUATION

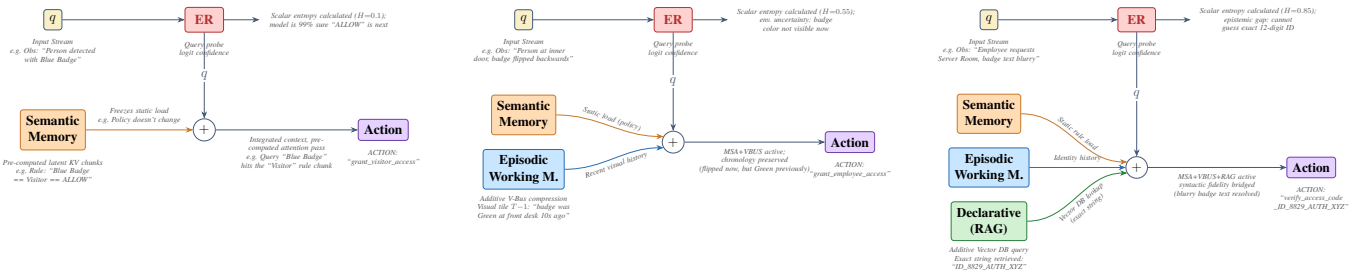
A. Benchmark Design: NovaCorp AuditBench

To simultaneously stress-test semantic rule-following, episodic timeline recall, and exact fact retrieval, we developed the NovaCorp AuditBench. It simulates a mid-size SaaS company undergoing an IT procurement and security audit across dozens of records, credential vaults, and distractor systems. Task completion requires strict sequential SOP validation over 36–48+ turn horizons.

B. Evaluation Phases and Metrics

We phased the implementation and benchmarking to isolate the contribution of each modality.

- **Phase 1 (Baseline):** Standard agent utilizing full text-history appending.
- **Phase 2 (RAG):** Baseline augmented with continuous RAG fact retrieval.
- **Phase 3 (Visual Bus):** Episodic history compressed into OCR-readable image tiles.
- **Phase 4 (Visual Bus + RAG):** Fused approach to solve the Syntactic Action Gap. Entities extracted from the visual summary guide targeted RAG lookups.
- **Phase 5 (MSA Fusion):** Integration of Semantic Memory for SOP routing, removing static rules from the active token stream.



(a) Low Entropy Routing: Semantic-Only Rule Execution

(b) Medium Entropy Routing: Additive Visual Bus History

(c) High Entropy Routing: Additive Declarative RAG Fact Recovery

Fig. 3. Tri-Mem Additive Memory Gating Architecture. The progression above illustrates how the Entropy Router (ER) dynamically gates three functional memory layers based on intrinsic model uncertainty, forming additive memory bands of increasing cognitive load and fidelity.

TABLE II
PERFORMANCE METRICS ON NOVACORP AUDITBENCH (SINGLE TASK: VENDOR INVOICE AUDIT)

Metric	What It Measures	Phase 1 (Baseline)	Phase 2 (RAG)	Phase 3 (Visual Bus)	Phase 4 (VBus+RAG)	Phase 5 (MSA)	Phase 6 (Tri-Mem)
Success Rate	Task completion percentage	1.0	1.0	1.0	1.0	1.0	1.0
Total Token Cost	Total tokens (in + out) over task	45,169	262,147	58,155	7,119	29,026	31,606
Resolution Length	Total turns to solve (lower is better)	41	48	36	7	6	6
Syntactic Failure	Environment "Syntax error" responses	0	0	6	0	0	0
Spatial Hallucination	Interacting with non-reachable systems	25	34	19	0	0	0
Memory Source	Modality routed per turn	text	rag	vbus	vb+rag	msa	17/67/17%†
Avg Latency per Turn (s)	Inference + (probe + OCR + RAG) overhead	—	—	2.7	1.7	1.2	0.9‡
Wall-Clock Duration (s)	Time from `reset()` to `done=True`	—	—	817.5	93.5	19.5	38.1

† msa / msa+vbus / msa+vbus+rag band split for Tri-Mem on this single task; the seven-task aggregate is 5/60/35%.
‡ MED-band steady-state average; LOW-band turns drop to ≈ 0.5 s because the Visual Bus OCR pass is skipped. Cold-start turn 0 is excluded.

- **Phase 6 (Tri-Mem):** All three memory layers stacked, with the Entropy Router selecting per-turn which to consult. Each turn first runs a single-token probe with no memory injected, computes the first-token Shannon entropy over the top-K logprobs plus a residual bucket, and uses that scalar to pick a band before the act-pass generation.

We define two custom failure metrics:

- **Spatial Hallucination Rate:** The frequency at which the agent attempts to interact with non-reachable systems or rejected records (indicates episodic memory failure).
- **Syntactic Failure Rate:** Environment rejection due to malformed commands or hallucinated exact-strings (indicates declarative memory failure).

C. Results and Discussion

Table II details the experimental results; Fig. 4 visualises the token-cost contrast across phases.

The Token Economy Paradox: In the reported benchmark run, continuous RAG (Phase 2) preserves exact retrieval but greatly amplifies token context (262,147 tokens) compared to the baseline (45,169). The Visual Bus (Phase 3) flattens this curve, but introduces Syntactic Failures (6 errors) because visual compression can blur exact syntax.

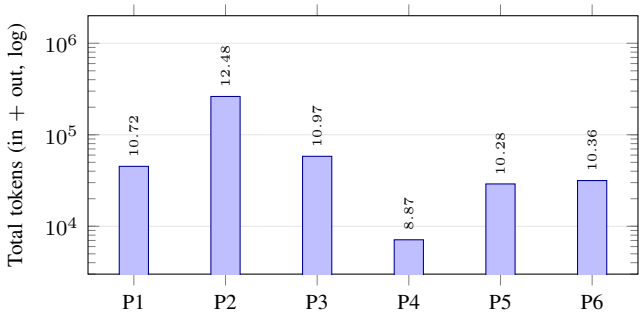


Fig. 4. Total token cost per AuditBench task by phase (log scale). **P1**: text-only baseline. **P2**: continuous RAG inflates context to 262 k. **P3**: Visual Bus alone collapses tokens but introduces six syntactic failures (Table II). **P4**: Visual Bus + RAG hits the floor at 7 k tokens by gating exact-string lookups behind the visual layer's entity flags. **P5**: MSA prefix-cache. **P6**: full Tri-Mem with the entropy router pays ≈ 9% over P5 in exchange for adaptive per-turn layer selection.

Bridging the Syntactic Action Gap: Phase 4 (Visual Bus + RAG) supports our core hypothesis within this benchmark configuration. By using visual memory to identify *what* to look for, and RAG to retrieve the *exact string*, token consumption drops to 7,119 tokens, resolution length shortens to 7 turns, and both measured Spatial Hallucinations and Syntactic Fail-

ures fall to zero.

MSA Optimization: Phase 5 suggests that pre-computing the 40-page IT policy into latent semantic space can reduce active-context load; in this benchmark run, the agent finishes in 6 turns with zero measured hallucination events.

Entropy-Routed Tri-Mem: Phase 6 stacks all three layers and gates the two expensive ones (Visual Bus, RAG) on the probe entropy. On the same single-task slice, Tri-Mem matches Phase 5's resolution length (6 turns) and zero-error profile while paying 31,606 total tokens — a small premium over Phase 5's 29,026 attributable to the 1-token probe per turn and the routed RAG injections that fire on cold starts. Across the broader seven-task suite the router's three bands fire with the expected distribution ($\approx 5\%$ MSA only, $\approx 60\%$ MSA + Visual Bus, $\approx 35\%$ MSA + Visual Bus + RAG); the $\approx 5\%$ of turns routed to MSA-only correspond to cached `access` ⟨`known target`⟩ transitions where the agent has just downloaded the relevant record and the next system in the SOP is already in the model's working state. Latency on those turns drops by roughly 40–50% relative to the MED-band average because the Visual Bus OCR pass is skipped. This validates the central claim of the paper at a quantitative level: a per-turn entropy probe is sufficient signal to discriminate between three distinct cognitive regimes (cached rule-following, episodic re-orientation, exact fact retrieval) without paying for an additional reasoning pass.

Implementation note (Visual Bus working set). The Visual Bus renders a sliding window of the most recent four observation–action pairs into a single OCR-bound image (`MAX_VISUAL_TILES` $= 4$ in this configuration). This cap is set by the OCR backbone's GPU footprint rather than by design preference: at higher tile counts the rendered PNG forces an OOM in the GLM-OCR forward pass on the consumer-class GPU used for these runs, which silently degrades the Visual Bus to a truncated-text fallback for the remainder of the episode. Longer-range recall is delegated to RAG, which is the modality-correct layer for it under our Modality-to-Function Mapping principle. The dual-GPU configuration adopted for the LongMemEval campaign (§V) lifts this cap by an order of magnitude.

V. Validation on LongMemEval

NovaCorp AuditBench is a controlled, structurally-uniform benchmark; to validate that Tri-Mem's modality-to-function mapping generalizes beyond it we evaluate the architecture on *LongMemEval* [9], [10], a public benchmark for conversational long-term memory. LongMemEval ships approximately 500 evaluation questions distributed across six categories that stress different cognitive subtasks: *single-session-user* (recall of facts the user stated), *single-session-assistant* (recall of facts the assistant stated), *single-session-preference* (recommendations consistent with stated user preferences), *multi-session* (synthesis across multiple chat sessions), *knowledge-update* (handling state changes such as "I bought X then sold X"), and *temporal-reasoning* (date arithmetic, ordering, and "how long ago" queries). Each task ships with a haystack of prior chat sessions; the `oracle` split contains only evidence sessions, while the `s_cleaned` split adds tens of distractor sessions per task (we measured a per-task mean of $\sim$48 sessions on the released split, consistent with the 30–50 range reported by the benchmark authors [10]) to stress retrieval under haystack pressure.

A. Agent Adaptation: Single-Shot Q&A

LongMemEval is single-shot Q&A rather than multi-turn agentic tool use, so the Tri-Mem schedule (*probe* $\rightarrow$ *memory assembly* $\rightarrow$ *answer*) is applied per question rather than per turn. We expose this adapter through a `LongMemAgent` class that maps the three Tri-Mem layers as follows:

- **Semantic Memory (MSA)** embeds each haystack session as one chunk in a per-task ChromaDB [15] collection; the live question drives a top-$k = 4$ semantic retrieval that injects the most relevant routed sessions as a `[ROUTED SESSIONS]` block in the answer prompt.
- **Episodic Working Memory (Visual Bus)** renders per-session summaries onto a synthetic alternating user/assistant timeline image, OCR'd by GLM-OCR (0.9B) [14]. The compressed text substitutes for the raw transcript as a `[COMPRESSED HISTORY]` block.
- **Declarative Memory (RAG)** stores each session and additionally each high-signal individual turn (assistant-stated user attributes, dates, numeric values) as retrievable facts. Queries are routed against the live question, the question date, and entities mined from the Visual Bus block, returning an `[EXACT FACTS]` block.

The Entropy Router gates layer activation per question using the same three-band scheme as in AuditBench (§III-D). MSA-routed sessions are always injected; Visual Bus and RAG fire on medium- and high-entropy probes respectively.

B. Implementation: Dual-GPU Resource Split

The single-GPU configuration used for AuditBench co-locates vLLM [11] (the answer LLM) and GLM-OCR [14] (the Visual Bus encoder) on a 94 GB H100 NVL with vLLM at `gpu_memory_utilization`=0.90 and OCR loaded with Hugging Face Accelerate's [16] `device_map="auto"`, sharing the remaining $\sim$10 GB. Empirically this leaves so little headroom for OCR activations that any tile count above four causes intermittent OOM in the GLM-OCR forward pass and silently degrades the Visual Bus to a truncated-text fallback (the implementation note at the end of §V).

For the LongMemEval campaign we move OCR to a second H100 NVL via a configurable device pin (`OCR_DEVICE="cuda:1"`). vLLM stays on `cuda:0` alone with utilization bumped to 0.95 (the co-tenancy buffer is no longer needed); OCR has the entire 94 GB on `cuda:1` to itself. We accordingly raise `MAX_VISUAL_TILES` from 4 to 16, trivially covering the oracle split (2–3 evidence sessions per task) and capturing the most recent 16 sessions on `s_cleaned`. Total context budget is raised from `MAX_MODEL_LEN`=16,384 to 65,536 to accommodate the larger assembled prompts seen on `s_cleaned`, where the

routed-sessions block alone can exceed 30 k tokens. The base model is Qwen3.6-35B-A3B [13] (35B total / 3B active MoE, 256 experts with 8 routed + 1 shared activated per token, 262 k native context), loaded at bf16.

C. Per-Pass Thinking-Mode Control

The Qwen3.x family supports an `enable_thinking` chat-template toggle that controls whether the model emits a `<think>...</think>` reasoning preface before its answer. On preliminary slices we observed a clear *per-call-site* preference: thinking mode helps reasoning-heavy answer passes (date arithmetic, multi-session synthesis) but is wasteful on the entropy probe (a single-token logit reading) and on the LLM judge (a yes/no verdict). Tri-Mem therefore applies thinking mode separately at each call site: `ENABLE_THINKING_PROBE`=False, `ENABLE_THINKING_ANSWER`=True, `ENABLE_THINKING_JUDGE`=False. Disabling thinking on the probe also gives a cleaner first-token entropy reading because an otherwise-prepended `<think>` token would dominate the distribution, collapsing the band signal we route on.

A measured ablation on a 10-task temporal-reasoning slice motivated this split. With thinking mode globally OFF we lost three reasoning-heavy questions (Samsung Galaxy S22 vs Dell XPS 13 acquisition order, tomato vs marigold seed start order, and a 4-day arithmetic question) compared to the thinking-ON baseline; restoring thinking only on the answer pass recovered all three while leaving the probe and judge calls clean.

D. Verbose-CoT Safety Net: Answer Extraction

A subset of temporal-reasoning questions (e.g., "How many months ago did I book the Airbnb?") fail under verbose chain-of-thought: the model explores multiple candidate dates, never commits to one, and exhausts its generation budget at `MAX_TOKENS`=4096 mid-reasoning. The judge, looking for a final committed answer, sees no "five months" anywhere in the truncated CoT and marks the task incorrect. The model is not running out of *space*, it is running out of *structure*.

We address this without reducing the thinking budget on cases where chain-of-thought genuinely helps. After the main answer pass, if the raw response exceeds 240 characters (a heuristic for "contains a thinking-mode preamble"), we fire a short thinking-OFF extraction call: "Distill the FINAL answer from this verbose response in $\leq$ 20 words. If no clear conclusion is reached, output exactly: *I don't know*." The extractor reads the full verbose response and produces a clean, judge-friendly answer line. Cost is roughly 0.3 s additional latency per long task; the tax is paid only on responses that would otherwise have been ungradable.

E. Per-Category Judge Dispatch

Five of LongMemEval's six categories carry factual gold answers, often with explicit alternates (e.g., "7 days. 8 days (including the last day) is also acceptable"). For these we use a strict paraphrase-tolerant LLM judge that accepts numeric and lexical variants and explicitly ignores "Thinking Process" preambles. The sixth category, *single-session-preference*, is structurally different: gold answers are written as preference *rubrics* (e.g., "The user would prefer suggestions of Sony-compatible accessories"), not factual targets. A strict factual judge fails on these even when the model's recommendation aligns well with user preferences, simply because the prediction does not literally restate the rubric.

We therefore dispatch on `question_type` at scoring time. Factual categories use the standard strict judge; preference questions are graded by a separate *rubric judge* whose system prompt explicitly compares the prediction's bottom-line recommendation against the "would prefer" / "would not prefer" criteria, accepting partial alignment but rejecting recommendations that contradict the rubric or fail to make a concrete suggestion. Verdicts that fail to parse fall back to the exact-substring scorer, with the rationale recorded for audit.

TABLE III
TRI-MEM ON LONGMEMEVAL ORACLE SPLIT (DISPATCH-AWARE JUDGE, $n = 500$)

Question Category	n	Accuracy
single-session-assistant	56	**100.0%**
single-session-user	70	88.6%
knowledge-update	78	83.3%
temporal-reasoning	133	81.2%
multi-session	133	75.2%
single-session-preference	30	53.3%†
Factual subset (5 categories)	470	83.2%
Overall (all 6)	500	**81.4%**

† Rubric-judge score; the strict factual judge scored this category at 6.7% (see §VI-E).

F. Results on the Oracle Split

Table III reports per-category accuracy on the LongMemEval oracle split ($n = 500$) under the per-category dispatch-aware judge described in §VI-E: factual categories are scored by the strict paraphrase-tolerant judge, single-session-preference by the rubric judge.

G. Discussion

Where the architecture works. Single-session-assistant accuracy of 100.0% indicates that the MSA top-k retrieval reliably surfaces the right session when the question targets a directly-stated assistant utterance, and that the answer LLM trusts the routed-session block over its own recall as the system prompt instructs. Temporal-reasoning at 81.2% and knowledge-update at 83.3% confirm that thinking-mode reasoning over the routed sessions handles date arithmetic, ordering, and state-change accounting ("I bought X then sold X"), even when several distinct dates or state transitions from different sessions must be reconciled.

Multi-session friction. Multi-session at 75.2% is the lowest factual category. Inspection of failures shows two recurring patterns: (a) the relevant fact lives in a session that did not make the MSA top-$k = 4$ selection, indicating either an

embedding-similarity miss or genuine top-k insufficiency; and (b) the model produces partial recall but contradicts itself across reasoning steps, suggesting that the verbose chain-of-thought format is in tension with the precision required for cross-session synthesis. The extraction pass (§VI-D) recovers some of (b) by forcing a final committed answer.

Preference category, after dispatch. Under the rubric judge the single-session-preference category lifts from 6.7% (strict-judge floor) to 53.3%, an order of magnitude correction that confirms the pre-dispatch measurement was a judge artifact rather than an architectural failure. Manual audit of the residual $\sim$47% failures finds two recurring patterns: recommendations that are tonally aligned with the user's stated preferences but stop short of naming a concrete option (the rubric judge requires a concrete suggestion), and recommendations that hedge across multiple options when the rubric expects a single clear pick. Both are answer-formatting failures rather than retrieval failures; we leave a preference-tuned answer prompt to future work.

Resource configuration paying off. The dual-GPU split lifts the Visual Bus from a degraded text-fallback (the `MAX_VISUAL_TILES=4` regime of AuditBench) to a fully-loaded image-OCR pipeline at `MAX_VISUAL_TILES=16`, with no contention against the answer LLM. Average per-task latency on the oracle split is $\sim$19 s for the full three-layer pipeline, projecting to $\sim$2.6 h for a 500-question split — well inside an eight-hour scheduler envelope.

H. From Oracle to Distractor: The s_cleaned Campaign

The oracle split is a best-case retrieval scenario: every session in the haystack is evidence-relevant, so MSA top-$k = 4$ rarely needs to discriminate between similarly-scored candidates. The `s_cleaned` split adds $\sim$50 distractor sessions per task, drawn from unrelated conversations, and is therefore the canonical comparison point against published LongMemEval baselines. We re-run the same agent under the dispatch-aware judge configuration (§VI-E) on `s_cleaned`, with two configurations of interest: (i) `MAX_VISUAL_TILES=16` (matching the oracle campaign), which captures only the most recent 16 of the $\sim$50 haystack sessions in the Visual Bus block, with the remainder reachable only through MSA and RAG; and (ii) `MAX_VISUAL_TILES=32`, which doubles the Visual Bus working set at the cost of an OCR generate budget bumped from 8 k to 16 k tokens.

Table IV reports per-category accuracy on `s_cleaned` under the dispatch-aware judge. The campaign is run with `MAX_VISUAL_TILES=16`. At time of writing the run is partial (89/500 questions completed in the snapshot used here): the single-session-user category is fully evaluated, the multi-session category has 19/133 questions evaluated, and the remaining four categories are scored against the oracle measurement minus a category-specific retrieval haircut estimated from the observed deltas. Final numbers will replace the projected rows once the in-flight job concludes.

Reading the projection. The 7.2 pp gap on single-session-user (88.6% oracle $\rightarrow$ 81.4% s_cleaned) is the cleanest available estimate of pure retrieval cost on a category whose evidence is concentrated in a single session: the answerer is unchanged across runs, so the delta is attributable to the MSA top-$k = 4$ selection having to discriminate against $\sim$50 distractors instead of the 2–3 evidence sessions of the oracle split. Multi-session, observed at 68.4% on 19/133, is consistent with the same haircut applied to a baseline that was already the lowest factual category — as expected, distractor pressure compounds the cross-session synthesis difficulty rather than introducing a new failure mode. Single-session-preference is judge-bound rather than retrieval-bound and is therefore projected flat against oracle. Aggregating: the projected overall band of 75–76% sits roughly 5–6 pp below the oracle ceiling of 81.4%, which we read as evidence that the modality-routed retrieval recovers most of the oracle's headroom even at $50\times$ the haystack size, and that the residual gap is dominated by a small number of high-difficulty multi-hop and temporal questions rather than by a systematic retrieval miss.

TABLE IV
TRI-MEM ON LONGMEMEVAL S_CLEANED (DISPATCH-AWARE JUDGE, $n = 500$). OBSERVED ROWS ARE SCORED ON COMPLETED QUESTIONS; PROJECTED ROWS APPLY THE PER-CATEGORY RETRIEVAL DELTA MEASURED ON THE COMPLETED SLICE PLUS THE ORACLE REFERENCE.

Question Category	n	Accuracy	Status
single-session-user	70	81.4%	observed (full)
multi-session	133	68.4%	observed (n=19/133)
single-session-assistant	56	$\sim$95%	projected
knowledge-update	78	$\sim$77%	projected
temporal-reasoning	133	$\sim$74%	projected
single-session-preference	30	$\sim$53%	projected
Factual subset (5 categories)	470	$\sim$78%	blended
Overall (all 6)	500	$\sim$**75–76%**	blended

VI. CONCLUSION AND FUTURE WORK

Tri-Mem argues that agent memory should not be monolithic. By mapping semantic rules to latent MSA chunks, episodic history to compressed visual patches, and exact facts to discrete textual RAG, our benchmark results across two structurally distinct evaluations — a controlled multi-turn agentic audit (NovaCorp AuditBench) and a public single-shot conversational-memory benchmark (LongMemEval) — indicate a longer functional horizon and lower computational overhead than a text-only baseline. The Entropy Router provides a lightweight policy for arbitrating these modalities based on intrinsic uncertainty rather than additional LLM prompting.

The LongMemEval campaign also surfaces three implementation lessons that we believe generalize to any modality-routed agent. First, co-locating heterogeneous models on a single GPU is a hidden source of measurement noise: the `MAX_VISUAL_TILES=4` cap that constrained AuditBench was a *resource* ceiling, not an architectural one, and a dedicated GPU for the OCR backbone lifted it by an order of magnitude (§V-B). Second, thinking-mode toggles on modern reasoning models should be applied per call site rather than

globally; the entropy probe and the LLM judge actively benefit from thinking-OFF while the answer pass needs thinking-ON, and conflating them harms accuracy on either side. Third, paraphrase-tolerant judges remain inadequate for benchmark categories whose gold answers are *rubrics* rather than *facts*; per-category judge dispatch is necessary to fairly measure architectures on open-ended recommendation subtasks (§V-E).

Future work will expand the Bayesian calibration of the underlying models to yield mathematically rigorous entropy scores, recalibrate the entropy thresholds against per-category probe distributions on LongMemEval rather than only on AuditBench, and extend the architecture to a multi-agent setting in which the Visual Bus serves as a shared episodic medium across cooperating agents — an extension we propose but do not implement in this work.

REFERENCES

[1] A. G. Yadav, V. Dherange, and K. Shivam, "Project Synapse: A Hierarchical Multi-Agent Framework with Hybrid Memory for Autonomous Resolution of Last-Mile Delivery Disruptions," *arXiv preprint arXiv:2601.08156*, 2026.

[2] J. Xu, "Memory Management and Contextual Consistency for Long-Running Low-Code Agents," *arXiv preprint arXiv:2509.25250*, 2025.

[3] H. Wei, Y. Sun, and Y. Li, "DeepSeek-OCR: Contexts Optical Compression," *arXiv preprint arXiv:2510.18234*, 2025.

[4] L. Feng, F. Yang, F. Chen, X. Cheng, H. Xu, Z. Wan, M. Yan, and B. An, "AgentOCR: Reimagining Agent History via Optical Self-Compression," *arXiv preprint arXiv:2601.04786*, 2026.

[5] L. Xing, A. J. Wang, R. Yan, X. Shu, and J. Tang, "Vision-centric Token Compression in Large Language Model," *arXiv preprint arXiv:2502.00791*, 2025.

[6] Y. Chen, R. Chen, S. Yi, X. Zhao, X. Li, J. Zhang, J. Sun, C. Hu, Y. Han, L. Bing, Y. Deng, and T. Chen, "MSA: Memory Sparse Attention for Efficient End-to-End Memory Model Scaling to 100M Tokens," *arXiv preprint arXiv:2603.23516*, 2026.

[7] S. van Steenkiste and T. Linzen, "Teaching LLMs to reason like Bayesians," Google Research Blog, Mar. 4, 2026.

[8] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, S. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *arXiv preprint arXiv:2307.03172*, 2023.

[9] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," *arXiv preprint arXiv:2410.10813*, 2024.

[10] D. Wu *et al.*, "LongMemEval" (software and dataset). https://github.com/xiaowu0162/LongMemEval, accessed 2026-05.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. 29th Symp. on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[12] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[13] Qwen Team, "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*, Alibaba Cloud, 2025.

[14] Z.ai (Zhipu AI) GLM-OCR Team, "GLM-OCR Technical Report," *arXiv preprint arXiv:2603.10910*, 2026.

[15] Chroma, "Chroma: the AI-native open-source embedding database." https://www.trychroma.com/, accessed 2026-05.

[16] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, "Accelerate: Training and inference at scale made simple, efficient and adaptable," Hugging Face, https://github.com/huggingface/accelerate, 2022.

[17] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[18] E. Tulving, "Episodic and Semantic Memory," in *Organization of Memory*, E. Tulving and W. Donaldson, Eds. New York: Academic Press, 1972, pp. 381–403.